



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه دوم درس طراحی الگوریتم

ساخت کابل‌های پشتیبان برای شبکه

استاد درس: دکتر رضا قنبری

اعضای گروه:

سارا مسگر مقدم، اسما موحدی تبار

فهرست مطالب

۳	۱ فاز سوم: یافتن بهینه‌ترین مجموعه کابل‌های پشتیبان
۳	۱.۱ مقدمه
۳	۲.۱ مدل‌سازی مسئله
۴	۳.۱ عدم تضمین بهینگی روش حریصانه
۵	۴.۱ ایده اصلی Bound and Branch
۶	۵.۱ ساختار حالت‌ها در درخت جستجو
۷	۶.۱ عملیات Branching
۷	۱.۶.۱ شاخه انتخاب کابل
۷	۲.۶.۱ شاخه عدم انتخاب کابل
۸	۷.۱ تابع کران پایین
۹	۸.۱ قانون هرس با استفاده از کران پایین
۹	۹.۱ اثبات عدم حذف جواب بهینه
۱۰	۱۰.۱ بررسی امکان‌پذیری یک حالت
۱۰	۱۱.۱ الگوریتم کلی
۱۱	۱۲.۱ پیچیدگی زمانی
۱۲	۱۳.۱ مقایسه فاز دوم و فاز سوم
۱۲	۱۴.۱ نمونه عددی مقایسه الگوریتم حریصانه و Bound and Branch
۱۴	۱.۱۴.۱ عملکرد الگوریتم حریصانه
۱۵	۲.۱۴.۱ عملکرد Bound and Branch
۱۶	۳.۱۴.۱ نتیجه مثال

۱۵.۱ نتیجه‌گیری ۱۶

۱ فاز سوم: یافتن بهینه‌ترین مجموعه کابل‌های پشتیبان

۱.۱ مقدمه

در فاز دوم، یک الگوریتم حریصانه برای انتخاب کابل‌های پشتیبان طراحی شد. این الگوریتم با توجه به یک معیار محلی، در هر مرحله کابلی را انتخاب می‌کرد که بیشترین بهبود را نسبت به هزینه آن ایجاد کند.

اگرچه این روش می‌تواند در زمان کوتاهی یک جواب قابل قبول ایجاد کند، اما انتخاب‌های حریصانه الزاماً به جواب بهینه منجر نمی‌شوند. زیرا تصمیم گرفته شده در هر مرحله تنها بر اساس وضعیت فعلی است و تأثیر انتخاب‌های آینده را در نظر نمی‌گیرد.

در این فاز هدف، یافتن مجموعه‌ای از کابل‌های پشتیبان با کمترین هزینه ممکن است؛ به گونه‌ای که پس از قطع شدن هر یک از یال‌های درخت پوشای کمینه، شبکه همچنان متصل باقی بماند.

برای رسیدن به جواب دقیق، از الگوریتم Branch and Bound استفاده شده است.

۲.۱ مدل‌سازی مسئله

فرض کنید درخت پوشای کمینه شبکه شامل مجموعه یال‌های زیر باشد:

$$E_{MST} = \{e_1, e_2, \dots, e_n\}$$

همچنین مجموعه کابل‌های پشتیبان برابر است با:

$$B = \{b_1, b_2, \dots, b_m\}$$

هر کابل پشتیبان b_i دارای:

- هزینه فعال‌سازی c_i
- مجموعه یال‌های MST که توسط آن پوشش داده می‌شوند

است.

برای هر کابل داریم:

$$Cover(b_i) \subseteq E_{MST}$$

هدف، انتخاب زیرمجموعه‌ای از کابل‌ها است:

$$S \subseteq B$$

به طوری که:

$$\bigcup_{b_i \in S} Cover(b_i) = E_{MST}$$

و هزینه کل کمینه شود:

$$\min \sum_{b_i \in S} c_i$$

بنابراین مسئله به یک نمونه از مسئله Weighted Set Cover تبدیل می‌شود.

۳.۱ عدم تضمین بهینگی روش حریصانه

روش حریصانه فاز دوم در هر مرحله کابلی را انتخاب می‌کند که بهترین نسبت سود به هزینه را در همان لحظه داشته باشد. اما این انتخاب محلی ممکن است باعث شود مجموعه نهایی هزینه بیشتری داشته باشد.

برای مثال فرض کنید سه یال MST وجود دارد:

$$E_{MST} = \{e_1, e_2, e_3\}$$

و کابل‌های زیر موجود باشند:

$$b_1 : Cover = \{e_1, e_2\}, \quad Cost = 5$$

$$b_2 : Cover = \{e_2, e_3\}, \quad Cost = 5$$

$$b_3 : Cover = \{e_1, e_2, e_3\}, \quad Cost = 11$$

در یک معیار حریصانه که تعداد یال پوشش داده شده نسبت به هزینه را بررسی می‌کند، ممکن است کابل b_1 انتخاب شود، زیرا:

$$\frac{2}{5} > \frac{3}{11}$$

پس از انتخاب b_1 ، برای پوشش e_3 نیاز به انتخاب b_2 خواهد بود:

$$Cost = 5 + 5 = 10$$

در حالی که انتخاب مستقیم b_3 تنها هزینه:

$$11$$

دارد.

(در نمونه‌های دیگر با تغییر هزینه‌ها می‌توان حالتی ساخت که انتخاب حریصانه حتی از جواب بهینه فاصله بیشتری داشته باشد.)

بنابراین روش حریصانه تضمین نمی‌کند که کم‌هزینه‌ترین زیرگراف دو-یال-همبند را تولید کند.

۴.۱ ایده اصلی Bound and Branch

الگوریتم پیشنهادی تمام انتخاب‌های ممکن کابل‌های پشتیبان را بررسی می‌کند، اما با استفاده از کران پایین و قوانین هرس، بخش‌هایی از فضای جستجو را که نمی‌توانند جواب بهتری تولید کنند حذف می‌کند.

هر گره درخت جستجو نشان‌دهنده یک تصمیم جزئی درباره کابل‌ها است. در هر مرحله یک کابل بررسی شده و دو شاخه ایجاد می‌شود:

۱. انتخاب کابل

۲. عدم انتخاب کابل

اگر کابل b_i بررسی شود، دو حالت داریم:

$Take(b_i)$

و

$Skip(b_i)$

شاخه اول:

$$Cost' = Cost + c_i$$

و مجموعه یال‌های پوشش داده شده به‌روزرسانی می‌شود.

شاخه دوم بدون افزایش هزینه، کابل فعلی را حذف کرده و کابل بعدی بررسی می‌شود.

۵.۱ ساختار حالت‌ها در درخت جستجو

برای نمایش هر گره در درخت جستجو، یک ساختار حالت (State) تعریف شده است. هر حالت شامل اطلاعات زیر است:

- **index:** شماره کابلی که در مرحله بعد بررسی خواهد شد.
- **covered:** یک آرایه بولی که مشخص می‌کند کدام یال‌های درخت پوشای کمینه توسط کابل‌های انتخاب شده تاکنون محافظت شده‌اند.
- **selected:** لیست کابل‌هایی که در مسیر فعلی درخت جستجو انتخاب شده‌اند.
- **cost:** مجموع هزینه کابل‌های انتخاب شده در این مسیر.

به صورت کلی یک حالت به شکل زیر نمایش داده می‌شود:

$$State = (index, covered, selected, cost)$$

ریشه درخت جستجو دارای وضعیت زیر است:

$$index = 0$$

$$cost = 0$$

و هیچ یالی از MST پوشش داده نشده است.

۶.۱ عملیات Branching

در هر مرحله، کابل شماره i بررسی می‌شود. دو فرزند برای گره فعلی ایجاد می‌شود.

۱.۶.۱ شاخه انتخاب کابل

در این حالت کابل فعال می‌شود:

$$selected' = selected \cup \{b_i\}$$

هزینه افزایش پیدا می‌کند:

$$cost' = cost + c_i$$

و تمام یال‌هایی که توسط این کابل پوشش داده می‌شوند، علامت‌گذاری می‌شوند:

$$covered[e] = true$$

سپس جستجو برای کابل بعدی ادامه پیدا می‌کند.

۲.۶.۱ شاخه عدم انتخاب کابل

در این حالت کابل موردنظر انتخاب نمی‌شود:

$$selected' = selected$$

$$cost' = cost$$

و تنها شماره کابل افزایش می‌یابد.

این تقسیم باعث می‌شود تمام زیرمجموعه‌های ممکن کابل‌ها در فضای جستجو وجود داشته باشند و در صورت عدم انجام هرس، جواب بهینه حتماً پیدا شود.

۷.۱ تابع کران پایین

برای کاهش تعداد حالت‌های بررسی شده، برای هر حالت یک کران پایین (Lower Bound) محاسبه می‌شود.

ایده اصلی این است که هزینه فعلی را با حداقل هزینه لازم برای پوشش یال‌های باقی‌مانده جمع کنیم.

فرض کنید:

$$C(s)$$

هزینه فعلی حالت s باشد.

برای هر یال MST که هنوز پوشش داده نشده است، کمترین هزینه کابل موجود که قادر به پوشش آن یال است را محاسبه می‌کنیم:

$$\minCost(e) = \min_{b_i: e \in Cover(b_i)} c_i$$

بنابراین کران پایین برابر است با:

$$LB(s) = C(s) + \sum_{e \notin covered(s)} \minCost(e)$$

این مقدار حداقل هزینه‌ای است که هر جواب کامل از این حالت می‌تواند داشته باشد.

۸.۱ قانون هرس با استفاده از کران پایین

اگر برای یک حالت داشته باشیم:

$$LB(s) \geq Best$$

که در آن:

$$Best$$

هزینه بهترین جواب کامل پیدا شده تاکنون است،

ادامه دادن این شاخه هیچ فایده‌ای ندارد؛ زیرا حتی در بهترین حالت نیز جوابی بهتر از جواب فعلی تولید نمی‌کند.

بنابراین این شاخه حذف می‌شود.

۹.۱ اثبات عدم حذف جواب بهینه

فرض کنید یک جواب بهینه OPT وجود دارد که از یک حالت s عبور می‌کند.

طبق تعریف کران پایین:

$$LB(s) \leq Cost(OPT)$$

زیرا کران پایین تنها یک تخمین خوش‌بینانه از هزینه ادامه مسیر است.

اگر شرط هرس برقرار باشد:

$$LB(s) \geq Best$$

با توجه به اینکه:

$$Best \leq Cost(OPT)$$

نتیجه می‌شود که:

$$LB(s) \geq Cost(OPT)$$

اما چون:

$$LB(s) \leq Cost(OPT)$$

داریم:

$$LB(s) = Cost(OPT) = Best$$

بنابراین حذف این شاخه باعث از دست رفتن جواب بهتر نمی‌شود.

در نتیجه الگوریتم Bound and Branch همچنان جواب بهینه را تضمین می‌کند.

۱۰.۱ بررسی امکان‌پذیری یک حالت

علاوه بر کران پایین، قبل از ادامه یک شاخه بررسی می‌شود که آیا هنوز امکان پوشش تمام یال‌های MST وجود دارد یا خیر.

برای این کار تمام کابل‌هایی که هنوز بررسی نشده‌اند بررسی شده و مجموعه یال‌هایی که در آینده می‌توانند پوشش داده شوند محاسبه می‌شود.

اگر حتی با استفاده از تمام کابل‌های باقی‌مانده، بعضی یال‌های MST بدون پوشش باقی بمانند، آن شاخه غیرممکن است و حذف می‌شود.

۱۱.۱ الگوریتم کلی

مراحل الگوریتم به صورت زیر است:

۱. یک حالت اولیه با هزینه صفر ایجاد می‌شود.

۲. کابل بعدی انتخاب شده و دو شاخه انتخاب و عدم انتخاب ایجاد می‌شود.

۳. پس از انتخاب کابل، هزینه و وضعیت پوشش به‌روزرسانی می‌شود.
۴. اگر تمام یال‌های MST پوشش داده شده باشند، جواب بررسی می‌شود.
۵. کران پایین محاسبه می‌شود.
۶. در صورت نامناسب بودن کران، شاخه هرس می‌شود.
۷. در پایان کمترین هزینه ثبت شده به عنوان جواب بهینه گزارش می‌شود.

۱۲.۱ پیچیدگی زمانی

در بدترین حالت، الگوریتم تمام زیرمجموعه‌های کابل‌ها را بررسی می‌کند.
تعداد زیرمجموعه‌ها برابر است با:

$$2^m$$

بنابراین پیچیدگی بدترین حالت:

$$O(2^m)$$

است.

اما در عمل به دلیل استفاده از:

- کران پایین
- حذف حالت‌های غیرممکن
- حذف مسیرهای دارای هزینه بیشتر از جواب فعلی

تعداد حالت‌های بررسی شده بسیار کمتر از جستجوی کامل خواهد بود.

۱۳.۱ مقایسه فاز دوم و فاز سوم

برای مقایسه، مثال زیر را در نظر می‌گیریم.

فرض کنید خروجی الگوریتم حریصانه فاز دوم کابل‌های زیر باشد:

$$\{b_1, b_2\}$$

با هزینه:

$$Cost = 15$$

اما الگوریتم Bound and Branch مجموعه:

$$\{b_3\}$$

را پیدا می‌کند:

$$Cost = 10$$

در این حالت روش حریصانه اگرچه شبکه را مقاوم می‌کند، اما هزینه بیشتری پرداخت می‌شود.

الگوریتم فاز سوم با بررسی سیستماتیک انتخاب‌ها توانسته است جواب کمینه را پیدا کند.

۱۴.۱ نمونه عددی مقایسه الگوریتم حریصانه و Bound and Branch

برای بررسی تفاوت عملکرد دو روش، شبکه‌ای شامل چهار شهر را در نظر می‌گیریم. فرض کنید درخت پوشای کمینه شبکه شامل سه یال زیر باشد:

$$E_{MST} = \{e_1, e_2, e_3\}$$

که به صورت زیر هستند:

$$e_1 = (1, 2)$$

$$e_2 = (2, 3)$$

$$e_3 = (3, 4)$$

سه کابل پشتیبان زیر در اختیار داریم:

$$b_1 = (1, 3), \quad Cost = 6$$

این کابل‌های زیر را پوشش می‌دهد:

$$Cover(b_1) = \{e_1, e_2\}$$

کابل دوم:

$$b_2 = (2, 4), \quad Cost = 6$$

که پوشش می‌دهد:

$$Cover(b_2) = \{e_2, e_3\}$$

و کابل سوم:

$$b_3 = (1, 4), \quad Cost = 10$$

که تمام کابل‌های درخت را پوشش می‌دهد:

$$Cover(b_3) = \{e_1, e_2, e_3\}$$

۱.۱۴.۱ عملکرد الگوریتم حریصانه

در روش حریصانه، معیار انتخاب کابل برابر نسبت تعداد یال‌های جدید پوشش داده شده به هزینه کابل در نظر گرفته می‌شود.

برای کابل‌ها داریم:

$$Score(b_1) = \frac{2}{6} = 0.33$$

$$Score(b_2) = \frac{2}{6} = 0.33$$

$$Score(b_3) = \frac{3}{10} = 0.3$$

بنابراین الگوریتم حریصانه ابتدا یکی از کابل‌های b_1 یا b_2 را انتخاب می‌کند.

فرض کنیم:

$$b_1$$

انتخاب شود.

در این حالت تنها یال e_3 بدون پوشش باقی می‌ماند، بنابراین کابل b_2 نیز باید انتخاب شود.

در نتیجه جواب الگوریتم حریصانه:

$$S_{Greedy} = \{b_1, b_2\}$$

و هزینه کل:

$$Cost(S_{Greedy}) = 6 + 6 = 12$$

خواهد بود.

۲.۱۴.۱ عملکرد Bound and Branch

الگوریتم Bound and Branch تمام حالت‌های ممکن انتخاب کابل‌ها را بررسی می‌کند.

یکی از حالت‌های بررسی شده:

$$S = \{b_3\}$$

است.

این مجموعه تمام یال‌های MST را پوشش می‌دهد:

$$Cover(b_3) = \{e_1, e_2, e_3\}$$

و هزینه آن برابر است با:

$$Cost(S) = 10$$

از آنجا که:

$$10 < 12$$

این جواب بهتر از جواب الگوریتم حریصانه است.

بنابراین جواب بهینه:

$$S_{OPT} = \{b_3\}$$

با هزینه:

$$Cost(S_{OPT}) = 10$$

خواهد بود.

۳.۱۴.۱ نتیجه مثال

در این مثال، الگوریتم حریصانه با انتخاب‌های محلی خود مجموعه‌ای با هزینه بیشتر تولید کرد، در حالی که Branch and Bound با بررسی فضای جواب و استفاده از هرس مناسب توانست جواب بهینه را پیدا کند.

جدول زیر مقایسه دو روش را نشان می‌دهد:

روش	کابل‌های انتخاب شده	هزینه نهایی
Greedy	$\{b_1, b_2\}$	۱۲
Bound and Branch	$\{b_3\}$	۱۰

این مثال نشان می‌دهد که معیارهای حریصانه تنها بر اساس اطلاعات محلی تصمیم می‌گیرند و ممکن است انتخابی انجام دهند که در ادامه باعث افزایش هزینه کل شود. در مقابل، الگوریتم Branch and Bound با بررسی انتخاب‌های آینده و استفاده از کران پایین، جواب کمینه را تضمین می‌کند.

۱۵.۱ نتیجه‌گیری

در این فاز، مسئله انتخاب کابل‌های پشتیبان به صورت یک مسئله بهینه‌سازی مدل شد. به دلیل وجود انتخاب‌های وابسته به آینده، روش‌های حریصانه تضمین رسیدن به جواب بهینه ندارند.

برای حل دقیق مسئله، الگوریتم Branch and Bound پیاده‌سازی شد. ساختار جستجوی دودویی تمام انتخاب‌های ممکن را مدل می‌کند و استفاده از کران پایین باعث حذف بخش بزرگی از فضای جستجو می‌شود.

در نهایت الگوریتم پیشنهادی ضمن حفظ تضمین بهینگی، در عمل نسبت به جستجوی کامل بسیار سریع‌تر عمل می‌کند.